Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

A

CASE-STUDY BASED REPORT

ON

<Spam Email Classification Using Naive Bayes and Support Vector Machine>

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Name KONDAMUDI HARIKA roll 2520030397

Name CHALLA AMRUTHA roll 2520030412

Name VESIREDDY ASHISH REDDY roll 2520030415

Under the Esteemed guidance of

Dr.Ravindra Gaddam

Asst.Professor, Dept. of CS&IT

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract **3**

Introduction 4

Background 4
Motivation for the Project 4
Overview of the Work Carried Out 5
Scope and Responsible-Use Statement 6

Problem Statement 7

Case-Study Scenario 7
Central Case-Study Question 7
Formal Problem Definition 8
Objectives that Guide the Study 8
Assumptions and Limitations 8

Dataset and Exploratory Data Analysis **10**

About the Dataset 10
Purpose of EDA 11
Preparation Steps Actually Performed 11
EDA Exploration - Implementation 12
Summary of EDA Findings 17

Algorithm Selection and Implementation 18

Conclusion **34**

Signature of the Team members

Abstract

Email spam remains a persistent threat that clutters inboxes, wastes bandwidth, and exposes users to phishing, malware, fraudulent messages and other online security risks. This case study develops an automated spam classification system that labels an incoming email as either *spam* or *legitimate (ham)* using two well-established supervised learning algorithms: **Naive Bayes** and the **Support Vector Machine (SVM)**. The central goal is an efficient and reliable filter that identifies unwanted mail accurately while keeping the number of legitimate emails wrongly sent to the spam folder as low as possible.

Raw email text is first cleaned with Natural Language Processing (NLP) techniques: normalization, tokenization, stop-word removal and stemming. The cleaned text is converted into numerical features with **Bag-of-Words** (word counts, used with Naive Bayes) and **Term Frequency–Inverse Document Frequency (TF-IDF)** (used with the SVM). The Naive Bayes classifier uses word-level probability distributions, while the linear SVM searches for the decision boundary that maximizes the margin between the two classes in a high-dimensional feature space.

Both models were trained and evaluated on the supplied labelled dataset of **10,000 emails** (3,995 spam and 6,005 ham, 20 columns). On a stratified 80/20 split (8,000 training and 2,000 test emails) **both models obtained 100% accuracy, precision, recall and F1-score** (zero misclassified test emails), and the same result held in 5-fold cross-validation. Additional experiments show that this perfect score is a property of the dataset, which is synthetic and built from fixed templates, rather than evidence that the problem is solved: when the 14 most obvious spam keywords were masked from the email body, accuracy fell to **88.0% for Naive Bayes and 98.8% for the SVM**, and the SVM was clearly the more accurate model when keywords were missing. On the full text, however, Naive Bayes tolerated flipped training labels better (it stayed at 100% with 30% of the training labels flipped, while the SVM fell to 87.2%) and it was slightly faster to predict.

The work is delivered as a complete system: a Python/scikit-learn back end (NLP pipeline, training and evaluation scripts, a Flask REST API) and a browser-based front end for live classification and model comparison. Throughout this report, numbers that were produced by running the project code are marked **ACTUAL RESULT**, and numbers or examples that are only meant to explain a concept or to show what might happen on other data are marked **ILLUSTRATIVE**.

Keywords: spam detection, text classification, Naive Bayes, Support Vector Machine, TF-IDF, Bag-of-Words, natural language processing, scikit-learn.

Introduction

Background

Electronic mail is one of the oldest and most widely used internet services, and for the same reasons it is a favourite channel for unwanted and malicious messages. *Spam* is usually defined as unsolicited bulk email. Besides being a nuisance, it delivers for phishing pages that imitate banks and online shops, for attachments carrying malware, for advance-fee and prize scams, and for misleading advertising. Every spam message costs network bandwidth and storage, and every minute a user spends sorting the inbox is lost working time. For organizations the largest risk is the single message that is opened by mistake.

Early defences were rule based: block lists of known sender addresses, hand-written keyword rules and header checks. Such rules are easy to understand but must be rewritten continually, because spammers change wording, misspell words on purpose and rotate sender domains. In the late 1990s researchers therefore began to treat spam filtering as a *text classification* problem that a program can learn from labelled examples. Sahami et al. (1998) proposed a Bayesian approach to junk e-mail filtering, Drucker, Wu and Vapnik (1999) compared support vector machines with other learners for spam categorization, and Paul Graham's essay *A Plan for Spam* (2002) made statistical filtering popular with practitioners. Naive Bayes and SVM have remained the standard baselines ever since.

A machine-learning spam filter has three layers. First, **NLP preprocessing** removes variation that does not help classification (letter case, punctuation, very common words, word endings). Second, **feature extraction** turns each message into a fixed-length numeric vector, typically word counts or TF-IDF weights. Third, a **classifier** learns from labelled training emails to separate the two classes and is then applied to new mail. This project implements exactly these layers and compares two classifiers that sit at opposite ends of the design spectrum: a simple probabilistic model (Naive Bayes) and a geometric large-margin model (SVM).

Motivation for the Project

Choosing a classifier for a spam filter is not only an accuracy question. A real mail system has to balance several demands at the same time, and the two algorithms studied here behave differently on each of them:

- **Cost of mistakes is asymmetric.** A spam message that reaches the inbox (false negative) is annoying, but a legitimate message that disappears into the spam folder (false positive) can

be a missed job offer or an important notice. A filter must therefore report precision and recall separately and not just overall accuracy.

- **Volume and speed.** A mail gateway may classify thousands of messages per minute, so training time, prediction time and memory matter. Naive Bayes needs only a single pass over the counts; the SVM solves an optimization problem.
- **Adaptation.** Spammers adapt, so the model has to be retrained regularly and should degrade gracefully when keywords are hidden or when some training labels are wrong.
- **Explainability.** An administrator and a user both want to know *why* a message was flagged. Both models used here are linear in the word features and therefore expose which words pushed a decision.

The team also wanted a practical, end-to-end learning experience for the Machine Learning course: starting from a raw CSV file, applying the NLP and vectorization techniques taught in class, training two classical algorithms, evaluating them with proper metrics, and finally exposing the trained models through a working web application rather than leaving them in a notebook.

Overview of the Work Carried Out

The work consisted of the following activities, each of which produced a concrete artifact that is described in the later chapters:

DATASET FACT | **Table 1.** Summary of the work carried out and the resulting artifacts.

#	Activity	Artifact / output	Where described
1	Dataset study and exploratory data analysis (EDA)	Class balance, feature distributions, data-quality checks, 8 figures	Dataset and EDA
2	NLP preprocessing pipeline	`preprocess.py`: normalization, tokenization, stop-word removal, stemming	Algorithm Selection
3	Feature extraction	Bag-of-Words and TF-IDF vectorizers (unigrams + bigrams)	Algorithm Selection
4	Model training	`train.py`: Multinomial Naive Bayes and Linear SVM, saved with joblib	Algorithm Selection
5	Evaluation	Accuracy, precision, recall, F1, AUC, confusion matrices, ROC, 5-fold CV	Algorithm Selection
6	Additional experiments	Ablations, keyword masking, metadata-only models, learning curves, label-noise test	Algorithm Selection
7	Web application	Flask REST API (`app.py`) and a three-tab browser front end	Algorithm Selection

All reported measurements were obtained by running the code in Python 3.13 with scikit-learn 1.9.1, pandas 3.0.5 and NumPy 2.5.3. The training script was also run on the team's own MacBook (Python 3.15.0b4, scikit-learn 1.9.1), where it printed the same metrics (accuracy,

precision, recall and F1 of 1.0000 for both models and the same confusion matrices), which confirms that the pipeline is reproducible.

Scope and Responsible-Use Statement

In scope. Binary classification (spam versus ham) of email *text*, using the subject and body, with two algorithms, two feature representations and a fixed preprocessing pipeline. The report covers data analysis, model building, evaluation, a demonstration web application and a critical discussion of what the results do and do not show.

Out of scope. Email headers beyond the supplied metadata columns, HTML parsing, images and attachments content, multilingual mail, adversarial attacks on the filter, deep-learning models, and deployment on a real mail server. The web application is a classroom demonstration and must not be used as the only protection for a real mailbox.

Responsible use. The dataset contains no real personal mail: the local parts of sender addresses are randomly generated strings and the text is generated from templates, so no private information is processed. If the approach were applied to real mail, the organization would need consent and a lawful basis, data minimization, and a review process for messages quarantined by mistake, because false positives can harm people.

How to read the results in this report: ACTUAL versus ILLUSTRATIVE

Every table and figure carries a small label in its caption. **ACTUAL RESULT** means the numbers were computed by running the project code on the supplied dataset (the scripts and the saved `results.json` can be re-run to reproduce them). **DATASET FACT** means a count or description read directly from the supplied CSV file.

ILLUSTRATIVE means the content is invented or simplified to teach a concept (for example a hand-calculated toy example or a hypothetical confusion matrix). Illustrative numbers were *not* obtained from this project's dataset and must not be quoted as findings. **SCHEMATIC** marks drawings of architecture or concepts that contain no results.

Problem Statement

Case-Study Scenario

Hypothetical scenario (for motivation only; no real organization or figures are implied). Consider the IT office of a university department that runs its own mail gateway. Staff and students receive a steady stream of messages: meeting reminders, schedule confirmations, project updates and weekly reports on one side, and unsolicited offers such as "WIN BIG NOW", "URGENT: claim your prize" or "You are selected" on the other. The existing blacklist catches some senders, but new sender domains appear constantly and the office cannot afford to maintain hand-written rules.

The office wants a content-based filter that learns from labelled examples. It must (a) keep legitimate academic mail flowing, (b) catch most of the spam, (c) be fast enough to run on modest hardware, and (d) be understandable, so that the administrators can explain to a professor why a message was quarantined. Two candidate algorithms are proposed, Naive Bayes and SVM, and the team is asked to build both, test them on a labelled sample and recommend one.

Central Case-Study Question

Central question

Given a labelled collection of emails, which of the two classifiers, Multinomial Naive Bayes on Bag-of-Words features or a linear Support Vector Machine on TF-IDF features, is more suitable for a practical spam filter, and what do the results tell us about how far these results can be trusted?

The question is broken down into the following sub-questions, which are answered in the Conclusion:

- **RQ1.** How accurately can each model separate spam from ham, and how many legitimate emails does each wrongly block?
- **RQ2.** Do the two feature representations (Bag-of-Words and TF-IDF) change the outcome?
- **RQ3.** How do the models compare in training time and prediction time?
- **RQ4.** How robust is each model when the obvious spam keywords are removed, when training data is small, or when some training labels are wrong?
- **RQ5.** Is the supplied dataset difficult enough to distinguish the two algorithms at all?

Formal Problem Definition

Let $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ be a set of n labelled emails, where x_i is the text of the i-th email (subject concatenated with body) and $y_i \in \{0, 1\}$ is its label (0 = ham, 1 = spam). The task is to learn a function

$$f : X \to \{0, 1\}, \qquad \hat{y} = f(\varphi(x))$$

where φ is a feature mapping that turns text into a numeric vector (here φ_BoW or φ_TF-IDF, both preceded by the preprocessing function g). The learning algorithm chooses f from a hypothesis family using only a training subset, and the quality of f is measured on a disjoint test subset that was not used for training. The objective is to maximize the F1-score of the spam class under the practical constraint that the false-positive rate (legitimate mail flagged as spam) stays as close to zero as possible. With TP, FP, TN and FN denoting the usual confusion-matrix counts for the spam class:

$$Precision = TP / (TP + FP), \qquad Recall = TP / (TP + FN), \qquad F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$$

Objectives that Guide the Study

- Understand the supplied dataset through exploratory data analysis and verify its quality before modelling.
- Build a reproducible NLP preprocessing pipeline (normalization, tokenization, stop-word removal, stemming).
- Represent emails numerically with Bag-of-Words and TF-IDF features.
- Train a Multinomial Naive Bayes classifier and a linear Support Vector Machine.
- Evaluate both with accuracy, precision, recall, F1-score, AUC, confusion matrices and cross-validation.
- Stress-test both models beyond a single train/test split and report honestly what the results mean.
- Provide a working front end and back end so that anyone can paste an email and see both predictions.

Assumptions and Limitations

Assumptions

- The labels in the supplied dataset are correct (they are treated as ground truth).

- Emails are independent and identically distributed, so a random split is representative of future mail from the same source.
- Subject and body text carry most of the information needed to classify the message.
- For Naive Bayes: word occurrences are conditionally independent given the class (the "naive" assumption).
- For the SVM: the two classes are approximately linearly separable in the TF-IDF feature space.

Limitations

- **Synthetic data.** The dataset appears to be machine-generated: spam bodies start with repeated template keywords, ham bodies with a fixed topic phrase, and the rest of each message is random filler words. Results therefore do *not* represent real-world difficulty (see the EDA chapter for evidence).
- **Single dataset, single language.** Only English text from one source was used; no real spam corpus (for example Enron or SpamAssassin) was available in this study.
- **Simple stemmer.** A lightweight suffix-stripping stemmer was implemented to avoid external downloads; it is less accurate than the Porter or Snowball stemmers.
- **Unused metadata.** The main models use only text. The 16 metadata columns were examined in EDA and tested separately, but are not part of the two main classifiers.
- **Timing measurements** were taken on a shared cloud machine and are indicative only.
- **No adversarial testing.** The filter was not tested against an attacker who deliberately crafts messages to evade it.

Dataset and Exploratory Data Analysis

About the Dataset

The dataset used in this study is the file `spam_email_dataset.csv` supplied for the project. It contains **10,000 emails** described by **20 columns**. Each row is one email with its subject, body text, a set of numeric and categorical metadata, and a label. The label column `label` is the target: 1 for spam (3,995 emails, 39.95%) and 0 for ham (6,005 emails, 60.05%). The classes are therefore mildly imbalanced, with about 40% spam and 60% ham.

Of the 20 columns, one is an identifier (`email_id`), one is the target (`label`) and **18 are potential input features**: two text columns that feed the NLP pipeline and 16 numeric, binary and categorical metadata columns. The two main classifiers in this study use only the two text columns, which is the setting described in the project abstract; the metadata columns are analysed in this chapter and evaluated separately in the next one.

DATASET FACT | **Table 2.** Schema of the supplied dataset (20 columns; read directly from the CSV file).

#	Column	Type	Meaning	Use in this study
1	email_id	int	Unique row identifier	Not a feature
2	subject	text	Email subject line	NLP → both models
3	email_text	text	Email body	NLP → both models
4	num_words	int	Word count of body	EDA, metadata test
5	num_characters	int	Character count of body	EDA, metadata test
6	num_exclamation_marks	int	Count of '!' in body	EDA (constant 0)
7	num_links	int	Number of links (0–8)	EDA, metadata test
8	has_suspicious_link	0/1	Suspicious link flag	EDA, metadata test
9	num_attachments	int	Attachments (0–2)	EDA, metadata test
10	has_attachment	0/1	Any attachment	EDA, metadata test
11	sender_email	category	Sender address (all unique)	EDA only
12	sender_domain	category	Domain of sender (8 values)	EDA, domain test
13	sender_reputation_score	float	Reputation, 0.3–1.0	EDA, metadata test
14	email_hour	int	Hour sent (0–23)	EDA, metadata test
15	email_day_of_week	int	Day (0–6)	EDA, metadata test
16	is_weekend	0/1	Weekend flag	EDA, metadata test
17	num_recipients	int	Recipients (1–30)	EDA, metadata test
18	contains_money_terms	0/1	Money-term flag	EDA, metadata test
19	contains_urgency_terms	0/1	Urgency-term flag	EDA, metadata test
20	label	0/1	Target: 1 = spam	Target

A first look at the text already shows its character. A typical spam email has the subject "ls WIN BIG NOW!!" and the body *"win free urgent offer limited limited urgent urgent free Beyond baby physical environmental none meeting your foreign low and."* A typical ham email has the subject "Weekly Report" and the body *"budget review - Statement our I claim world star. Myself a entire. Enough sport baby every wear economic daughter."* In both cases the second half of the body is a random sequence of ordinary English words, which looks like filler added by a text generator. This observation is important for interpreting the model results later.

Purpose of EDA

Exploratory Data Analysis is performed before modelling for four reasons. First, to check **data quality**: missing values, duplicates, constant or inconsistent columns. Second, to understand the **class balance**, because it affects the choice of metrics. Third, to discover which features carry **discriminative signal**, which guides what the classifiers can be expected to learn. Fourth, to look for **leakage and artefacts**, meaning features that make the task artificially easy and would make the evaluation over-optimistic. The last point turned out to be the most important finding of this study.

Preparation Steps Actually Performed

The following preparation steps were carried out in code (scripts `train.py` and `experiments.py`). Their outcomes are reported in Tables 3 and 4.

1. **Loading and inspection.** The CSV was read with pandas; shape, data types, missing values and duplicates were checked.
2. **Text assembly.** Subject and body were concatenated into one string per email (`subject + " " + email_text`).
3. **NLP preprocessing.** Each string was lower-cased, links replaced by the token `url`, digits by `num`, split into alphabetic tokens of two or more letters, stop words removed (scikit-learn's English list of 318 words) and each remaining token stemmed.
4. **Stratified split.** The data were split 80/20 into training and test sets with `random_state=42` and stratification on the label so that both parts keep the 40/60 class ratio.
5. **Fitting on training data only.** The vectorizers (vocabulary, document frequencies) were fitted on the training split only, which prevents information from the test set leaking into the features.

ACTUAL RESULT | **Table 3.** Data-quality checks performed on the CSV file.

Check	Result	Comment
Rows × columns	10,000 × 20	As documented above
Missing values	0	No imputation needed
Duplicate (subject, body) pairs	0	Every email text is unique
Attachments consistent	100%	has_attachment = (num_attachments > 0) in every row
Word/character counts consistent	100%	num_words and num_characters match the actual body text
is_weekend consistent	100%	Equals (day_of_week ≥ 5) in every row
Constant column	num_exclamation_marks	Always 0 (total 0), although several spam subjects contain '!': no information
Inconsistent flag	190 rows	has_suspicious_link = 1 while num_links = 0

Check	Result	Comment
Unique sender_email values	10,000	One per row: unusable as a feature

ACTUAL RESULT | **Table 4.** Effect of the preprocessing pipeline on one email. Over the whole dataset the average number of tokens per email fell from 20.50 to 16.42 (19.9% fewer), and the number of distinct unigrams from 985 to 765.

Stage	Example (email #2, a spam email)
Raw text	ls WIN BIG NOW!! win free urgent offer limited limited urgent urgent free Beyond baby physical environmental none meeting your foreign low and.
After normalization + tokenization (22 tokens)	win big now win free urgent offer limited limited urgent urgent free beyond baby physical environmental none meeting your foreign low and
After stop-word removal (17 tokens)	win big win free urgent offer limited limited urgent urgent free baby physical environmental meeting foreign low
After stemming (final input to vectorizer)	win big win free urgent offer limit limit urgent urgent free baby physical environmental meet foreign low

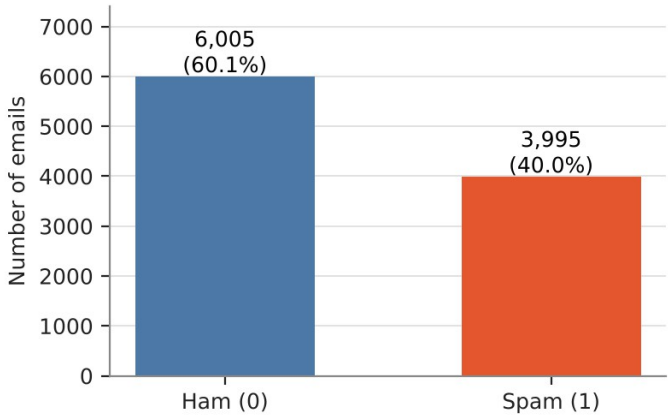
EDA Exploration - Implementation

The EDA was implemented with pandas and matplotlib. A short excerpt shows the style of the analysis (group statistics by class, then plots):

```
df = pd.read_csv("spam_email_dataset.csv")
print(df.shape, df.isna().sum().sum(), df.duplicated(["subject","email_text"]).sum())
print(df.groupby("label").mean(numeric_only=True).T)       # class-wise feature means
print(pd.crosstab(df.subject, df.label))                    # subject x class
print(pd.crosstab(df.sender_domain, df.label))              # domain x class
df[num_cols + ["label"]].corr()["label"]                    # correlation with target
```

Class balance

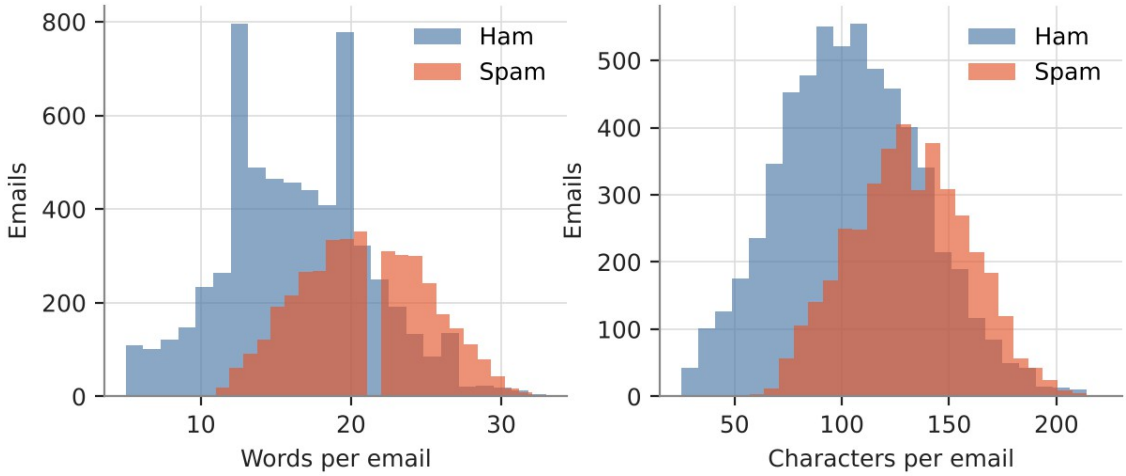
Figure 1 shows the class distribution. With 40.0% spam, a classifier that always answers "ham" would already reach 60.1% accuracy, so accuracy alone is not enough and precision, recall and F1 are reported too.



DATASET FACT | **Figure 1.** Class distribution of the 10,000 emails.

Length of the emails

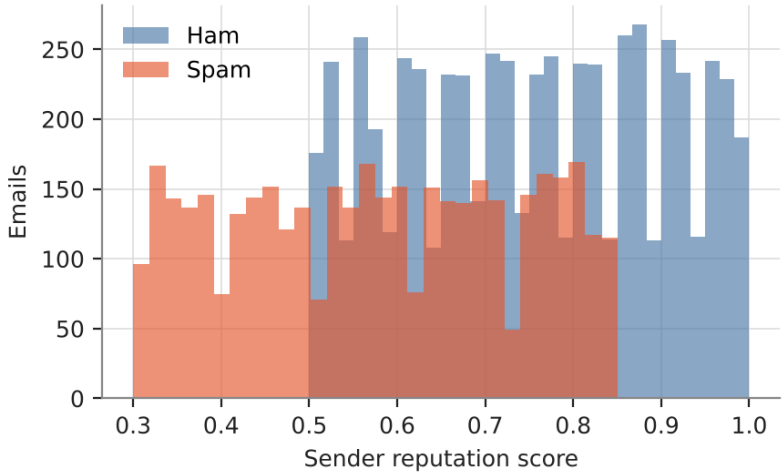
Spam emails are longer: on average 20.81 words and 131.3 characters, against 16.30 words and 104.3 characters for ham (Figure 2). The correlation of word count with the label is +0.43. The reason is mechanical: spam bodies start with a run of template keywords before the filler text, whereas ham bodies start with a short topic phrase.



ACTUAL RESULT | **Figure 2.** Distribution of words and characters per email, by class.

Sender reputation

The sender reputation score is the strongest numeric signal (correlation -0.50). The mean is 0.752 for ham and 0.577 for spam, but the distributions overlap strongly (ham ranges from 0.50 to 1.00, spam from 0.30 to 0.85), so the score alone cannot separate the classes (Figure 3).

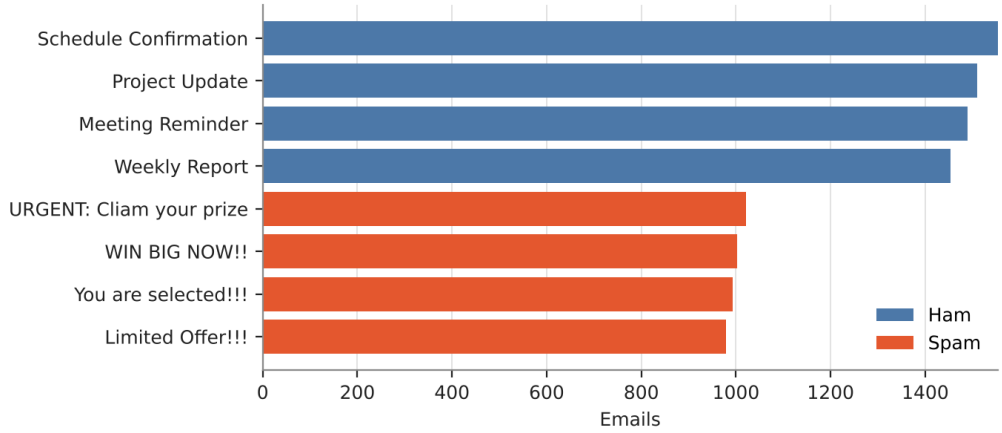


ACTUAL RESULT | **Figure 3.** Sender reputation score by class.

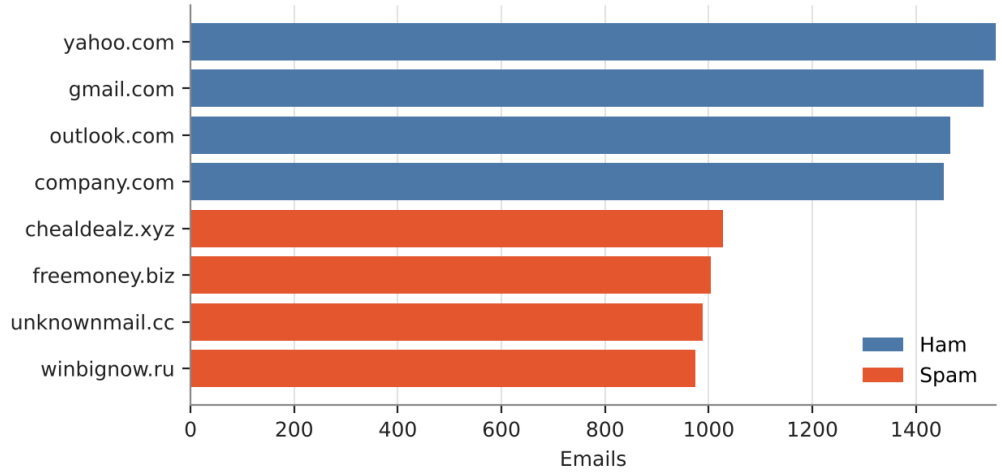
Subject lines and sender domains

Figures 4 and 5 reveal the most important property of the dataset. There are only **eight distinct subject lines** and **eight distinct sender domains**, and every one of them belongs to exactly one

class. The four ham subjects (Meeting Reminder, Project Update, Schedule Confirmation, Weekly Report) never occur in spam, and the four spam subjects ("ls WIN BIG NOW!!", "Limited Offer!!!", "URGENT: Cliam your prize", "You are selected!!!") never occur in ham. The same holds for the domains: ham comes only from company.com, gmail.com, outlook.com and yahoo.com, and spam only from chealdealz.xyz, freemoney.biz, unknownmail.cc and winbignow.ru.



DATASET FACT | **Figure 4.** Emails per subject line, split by class (each subject occurs in only one class).



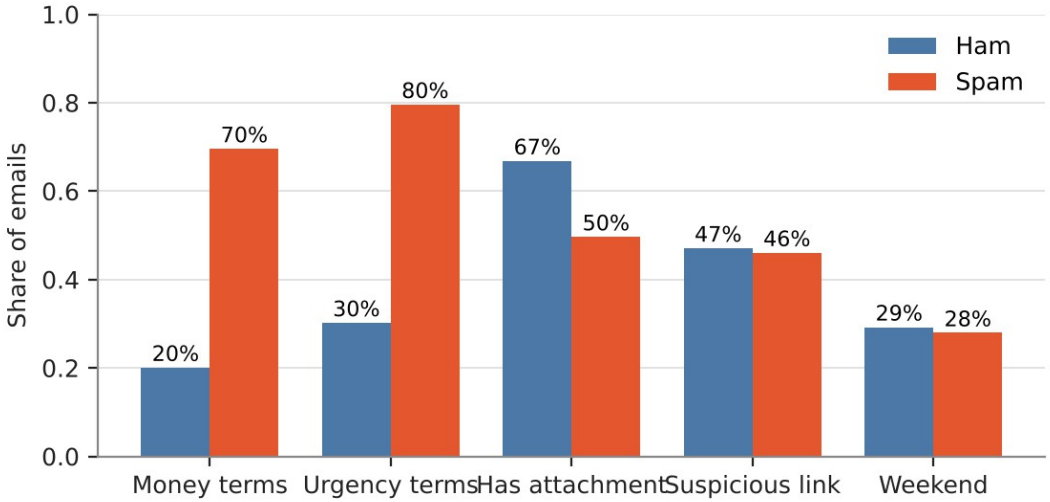
DATASET FACT | **Figure 5.** Emails per sender domain, split by class (each domain occurs in only one class).

A model given the subject line, or the sender domain, can therefore reach 100% accuracy by simply memorizing eight values. This is a case of **target leakage through artefacts of data generation**: in real email no sender domain or subject line is exclusive to one class.

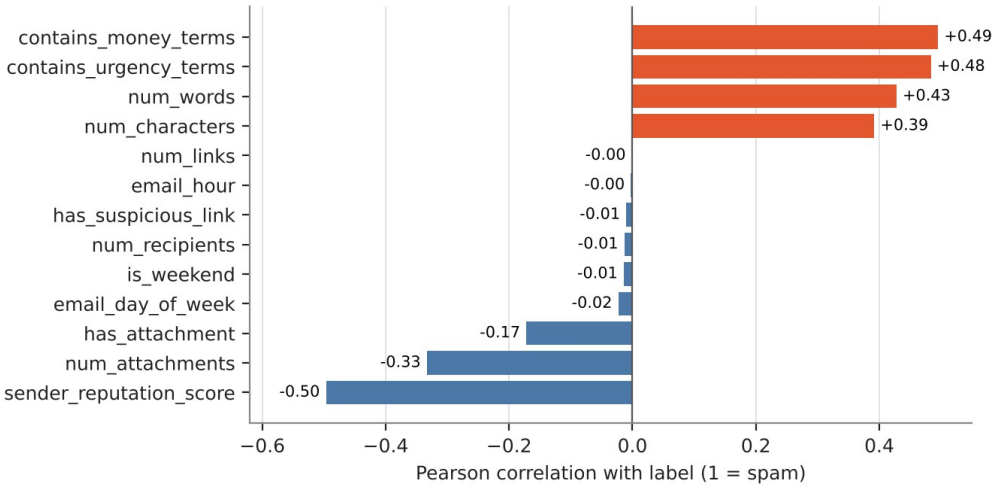
Binary flags and attachments

The flags `contains_money_terms` and `contains_urgency_terms` are set far more often in spam (69.6% and 79.6%) than in ham (20.1% and 30.2%), but they are noisy: 1,216 spam emails have no money flag and 1,206 ham emails have one. Attachments behave unexpectedly: ham

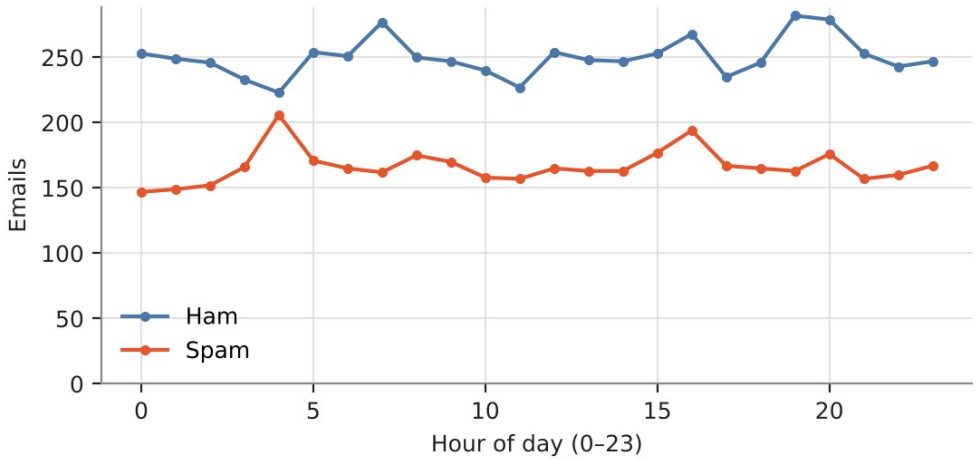
emails carry on average 1.01 attachments and spam 0.50; in the data, emails with two attachments are *never* spam. Links, weekend, suspicious-link flag, recipients and the hour of sending are practically identical in both classes (Figures 6 to 8).



ACTUAL RESULT | **Figure 6.** Share of emails with each binary flag, by class.



ACTUAL RESULT | **Figure 7.** Pearson correlation of each numeric feature with the spam label.



ACTUAL RESULT | **Figure 8.** Number of emails per hour of the day, by class. Both classes are almost uniform over the day.

ACTUAL RESULT | **Table 5.** Class-wise means and correlation with the label for selected numeric features.

Feature	Ham mean	Spam mean	Correlation with label
num_words	16.30	20.81	+0.428
num_characters	104.3	131.3	+0.392
sender_reputation_score	0.752	0.577	-0.497
num_attachments	1.008	0.496	-0.333
contains_money_terms	0.201	0.696	+0.495
contains_urgency_terms	0.302	0.796	+0.484
num_links	3.978	3.973	-0.001
has_suspicious_link	0.471	0.461	-0.010
num_recipients	15.593	15.363	-0.013

Summary of EDA Findings

- The dataset is clean in the usual sense (no missing values, no duplicates, consistent counts) with two small defects: the constant `num_exclamation_marks` column and 190 rows where a suspicious link is flagged although the link count is zero.
- Classes are mildly imbalanced (40% spam / 60% ham), so precision, recall and F1 are reported together with accuracy.
- Subject line and sender domain each separate the classes **perfectly**, because every value occurs in exactly one class. This is an artefact of synthetic data generation and is the main reason a text classifier will find the task trivial.
- Spam bodies begin with repeated template keywords (win, free, urgent, offer, limited, click, cash, guarantee, claim, prize, selected) and ham bodies with a fixed topic phrase (report, schedule, team sync, meeting, budget review, project update); the rest of each body is random filler.

- Among numeric metadata, sender reputation, number of attachments, word count and the two term flags carry partial signal; links, recipients, weekday and hour carry essentially none.
- **Implication.** Very high scores from the two classifiers are expected, and the evaluation must be complemented by stress tests (keyword masking, small training sets, noisy labels) to say anything about the relative merits of Naive Bayes and SVM. These are presented in the next chapter.

Algorithm Selection and Implementation

Candidate Algorithms and Selection Rationale

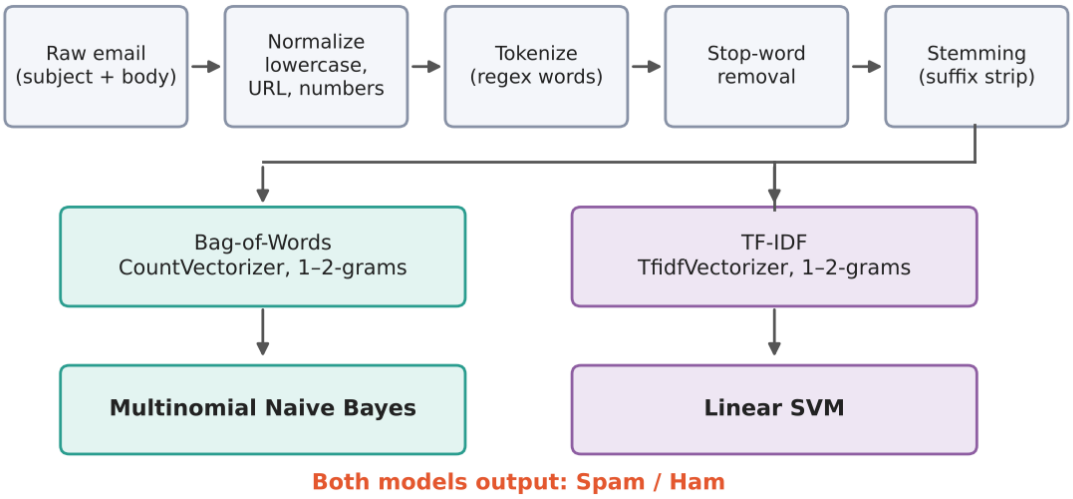
Many classifiers have been applied to spam filtering: Naive Bayes, logistic regression, SVM, decision trees and random forests, k-nearest neighbours and, more recently, neural networks. The project brief fixes the pair to be studied, Naive Bayes and SVM, and the literature supports this choice. Both work well with the very high-dimensional, sparse vectors produced by text (thousands of word features and only a few dozen non-zero values per email), both train in seconds on ten thousand emails, and both are linear in the features, which makes them explainable. Metsis et al. (2006) compare several Naive Bayes variants for spam filtering, Drucker et al. (1999) and Joachims (1998) show that SVMs are strong text classifiers, and McCallum and Nigam (1998) explain why the multinomial event model is the natural choice for word counts.

SCHEMATIC | **Table 6.** Conceptual comparison of the two selected algorithms (no experimental results).

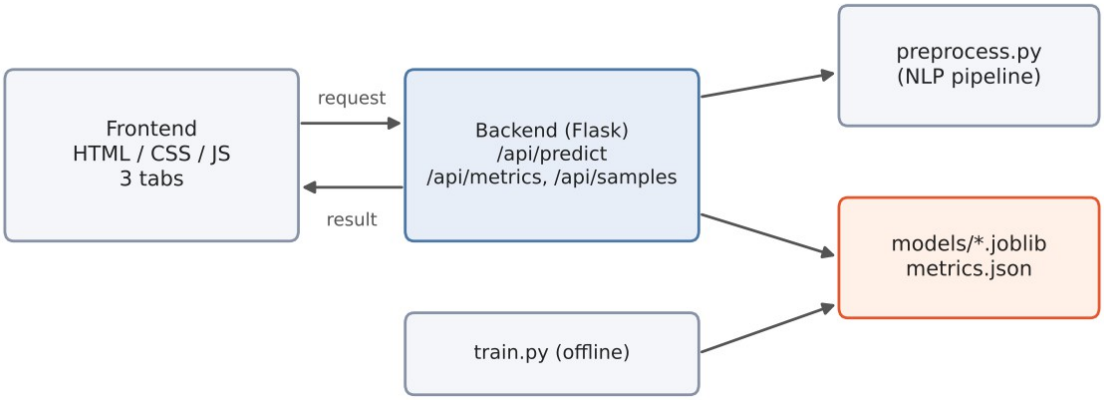
Property	Multinomial Naive Bayes	Linear SVM
Idea	Generative: models P(word | class) and applies Bayes' rule	Discriminative: finds the maximum-margin separating hyperplane
Feature representation used	Bag-of-Words counts (1–2-grams)	TF-IDF weights (1–2-grams, sublinear TF)
Training	Counting; one pass over data	Convex optimization (hinge loss, L2 penalty)
Key hyper-parameter	Smoothing α (0.5)	Regularization C (1.0)
Output	Class probabilities (often over-confident)	Signed distance to boundary; probabilities need calibration
Strengths	Very fast, works with little data, simple to update	Strong on high-dimensional sparse data, robust margin
Weaknesses	Independence assumption; sensitive to correlated or missing words	Slower to train, needs scaling of features and tuning of C

System Design

The system has two parts that share the same preprocessing code. In the **offline** part, `train.py` reads the dataset, applies the NLP pipeline, fits both models and writes the trained objects and the evaluation metrics to disk. In the **online** part, the Flask back end loads those files and answers requests from the browser: when a user submits an email, the back end preprocesses it with exactly the same function that was used for training, vectorizes it with the stored vectorizer and returns both predictions as JSON.



SCHEMATIC | **Figure 9.** Processing pipeline from raw email to prediction.



SCHEMATIC | **Figure 10.** Front-end / back-end architecture of the web application.

NLP Preprocessing Implementation

The preprocessing module implements the four techniques named in the project abstract. **Normalization** lower-cases the text and replaces every URL by the token `url` and every number by `num`, so that the model learns that an email contains a link rather than memorizing one particular address. **Tokenization** extracts alphabetic tokens of at least two letters with a regular expression, which also discards punctuation and emoji. **Stop-word removal** deletes frequent function words using scikit-learn's English list of 318 words. **Stemming** reduces words to a crude stem by stripping common suffixes (-ing, -ed, -ly, -es, -s), so that *claiming*, *claimed* and *claims* all map to *claim*; the rule keeps at least three letters of the stem.

```
ENGLISH_STOP = set(ENGLISH_STOP_WORDS)
SUFFIXES = ("ingly", "edly", "ing", "ed", "ly", "ies", "es", "s")
```

```
def stem(word):
    for suf in SUFFIXES:
        if word.endswith(suf) and len(word) - len(suf) >= 3:
            return word[:-len(suf)] + ("y" if suf == "ies" else "")
    return word

def tokenize(text):
    text = str(text).lower()                        # normalization
    text = re.sub(r"https?://\S+", " url ", text)  # links  -> 'url'
    text = re.sub(r"\d+", " num ", text)            # digits -> 'num'
    return re.findall(r"[a-z]{2,}", text)           # tokenization

def preprocess(text):
    return " ".join(stem(t) for t in tokenize(text) if t not in ENGLISH_STOP)
```

The stemmer is intentionally simple. It makes some harmless errors (for example the misspelling *cliam*, used by the spammer templates, is left alone) but it requires no external data download, which keeps the project easy to install. A Porter or Snowball stemmer from NLTK could replace it with a one-line change.

Feature Extraction: Bag-of-Words and TF-IDF

Machine-learning algorithms need numbers, so each preprocessed email is mapped to a sparse vector with one dimension per vocabulary entry. In this project the vocabulary contains **unigrams and bigrams** (single words and pairs of adjacent words) that occur in at least two training emails (`min_df=2`). Bigrams matter because phrases such as "win big", "limit offer" or "claim prize" are more specific than their individual words.

Bag-of-Words (BoW) records how many times each term occurs in the email: x_j = count(term j, email). It ignores word order and is the natural input for the multinomial Naive Bayes model. **TF-IDF** rescales the counts so that terms that appear in almost every email carry little weight and terms that are rare but concentrated in a few emails carry more:

$$tf\text{-}idf(t, d) = tf(t, d) \times idf(t), \quad idf(t) = \ln[(1 + n) / (1 + df(t))] + 1 \qquad (1)$$

Here n is the number of training emails and df(t) the number of those emails containing term t. The implementation uses sublinear term frequency (tf = 1 + ln count), which prevents a word that is repeated many times from dominating, and L2-normalizes each vector so that long and short emails are comparable. Both vectorizers learn a vocabulary of the same size from the training split:

ACTUAL RESULT | **Table 7.** Feature space produced by the vectorizers (fitted on the 8,000 training emails).

Item	Value
Vocabulary size (unigrams + bigrams, min_df = 2)	6,857
of which unigrams / bigrams	765 / 6,092
Feature vectors	Sparse matrices; about 16 non-zero terms (before bigrams) per email

```
bow   = CountVectorizer(ngram_range=(1, 2), min_df=2)                        # Bag-of-
Words
tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)    # TF-IDF
Xb = bow.fit_transform(X_train)        # fit on training data only
Xt = tfidf.fit_transform(X_train)
```

Naive Bayes Classifier

Naive Bayes applies Bayes' theorem to estimate the probability that an email *d* belongs to class *c* B {ham, spam}:

$$P(c \mid d) = P(d \mid c) \cdot P(c) / P(d) \;\acute{u}\; P(c) \cdot \prod_{\ddot{v}} P(w_{\ddot{v}} \mid c)^{\{x_{\ddot{v}}\}} \qquad (2)$$

The "naive" step is the assumption that the words of an email are independent given its class, which turns the joint probability of the email into a product of per-word probabilities. For the multinomial model the per-word probabilities are estimated from training counts with additive (Laplace/Lidstone) smoothing α, so that a word never seen in spam does not make the whole product zero:

$$P(w_{\ddot{v}} \mid c) = (N_{\ddot{v}c} + \alpha) / (N_c + \alpha \cdot |V|) \qquad (3)$$

where NΩc is the total count of term *j* in training emails of class *c*, Nc the total count of all terms in class *c*, and |V| the vocabulary size. In practice the computation is done in log space (sums of log-probabilities) to avoid numerical underflow, and the predicted class is the one with the larger log-posterior. This project uses `MultinomialNB(alpha=0.5)`. The class priors learned from the training split are P(ham) = 0.600 and P(spam) = 0.400.

Worked toy example (ILLUSTRATIVE). To make the mechanism concrete, take a tiny invented corpus of three spam emails ("win cash now", "free cash prize", "win free prize now") and three ham emails ("team meeting today", "project update meeting", "budget review today"), a vocabulary of 12 words, α = 1 and equal priors. The spam class contains 10 word tokens and the ham class 9. For the new email "free cash meeting" the smoothed probabilities are P(free|spam) = (2+1)/(10+12) = 0.136, P(cash|spam) = 0.136, P(meeting|spam) = (0+1)/22 = 0.045, and P(free|ham) = (0+1)/(9+12) = 0.048, P(cash|ham) = 0.048, P(meeting|ham) = (2+1)/21 = 0.143. Multiplying with the priors gives $0.5 \times 0.136 \times 0.136 \times 0.045 = 4.2\times10^{-4}$ for spam and $0.5 \times 0.048 \times 0.048 \times 0.143 = 1.6\times10^{-4}$ for ham, so the normalized spam probability is about **0.72** and the email is classified as spam, even though it contains one ham-like word.

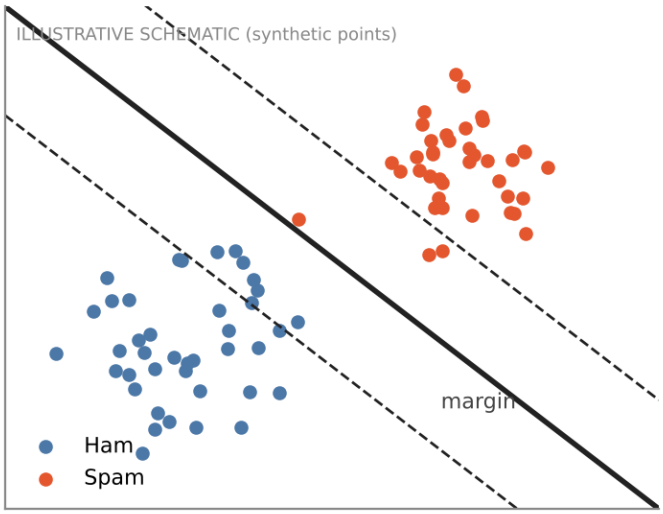
```
nb = MultinomialNB(alpha=0.5).fit(bow.transform(X_train), y_train)
p_spam = nb.predict_proba(bow.transform(X_test))[:, 1]
# explanation: log P(w|spam) - log P(w|ham) for every vocabulary term
w = nb.feature_log_prob_[1] - nb.feature_log_prob_[0]
```

Support Vector Machine Classifier

A Support Vector Machine represents each email as a point x in the TF-IDF space and looks for the hyperplane $w \cdot x + b = 0$ that separates spam from ham with the **largest margin**, i.e. the greatest distance to the nearest training points of both classes (the support vectors). Because real data are rarely perfectly separable, the soft-margin formulation allows some violations and trades margin width against errors through the regularization parameter C:

$$\text{minimize} \quad \tfrac{1}{2}\|w\|^2 + C \cdot \Sigma_i \max(0,\ 1 - y_i(w \cdot x_i + b)) \qquad (4)$$

The first term widens the margin, the second (hinge loss) penalizes points on the wrong side or inside the margin. A **linear kernel** is the standard choice for text, since sparse vectors with thousands of dimensions are already easy to separate linearly; scikit-learn's `LinearSVC` solves the problem efficiently. The sign of the decision value $w \cdot x + b$ gives the class. An SVM does not output probabilities, so the application wraps it in `CalibratedClassifierCV` (5-fold sigmoid calibration) to display a spam probability; the metrics reported in this chapter use the plain `LinearSVC` decision values and predictions. The weight w_j of a term shows how strongly it pushes a decision towards spam (positive) or ham (negative).



SCHEMATIC | **Figure 11.** Maximum-margin idea in two dimensions. The points are synthetic and drawn only to explain the concept; real emails live in a space with 6,857 dimensions.

```
svm = CalibratedClassifierCV(LinearSVC(C=1.0), cv=5)     # in the web app (adds
probabilities)
svm.fit(tfidf.transform(X_train), y_train)
# metrics in this report: plain LinearSVC(C=1.0).fit(...).predict(...)
```

Experimental Setup and Evaluation Metrics

DATASET FACT | **Table 8.** Experimental configuration used for every result marked ACTUAL.

Setting	Value
Data	10,000 emails; subject + body; label as target
Split	80% train (8,000; 3,196 spam) / 20% test (2,000; 799 spam), stratified, random_state = 42
Preprocessing	Normalize → tokenize → stop-words → stemming (identical for both models)
Naive Bayes	CountVectorizer(1–2-grams, min_df=2) + MultinomialNB(α = 0.5)
SVM	TfidfVectorizer(1–2-grams, min_df=2, sublinear_tf) + LinearSVC(C = 1.0)
Cross-validation	5-fold stratified, shuffled, random_state = 42
Software	Python 3.13, scikit-learn 1.9.1, pandas 3.0.5, NumPy 2.5.3, SciPy 1.18.1

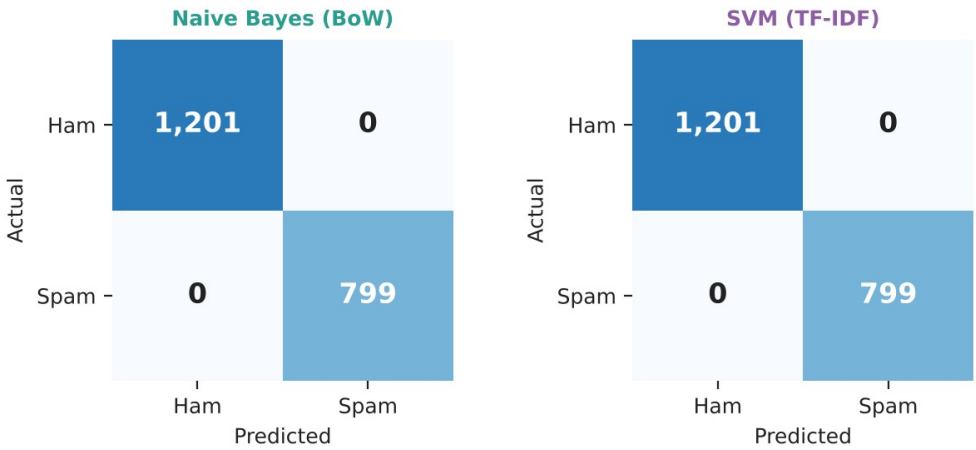
The positive class is spam. **Accuracy** is the share of all emails classified correctly. **Precision** is the share of emails flagged as spam that really are spam; its complement is the false-positive burden on legitimate mail. **Recall** is the share of all spam that is caught. **F1** is the harmonic mean of precision and recall. The **ROC curve** plots the true-positive rate against the false-positive rate over all decision thresholds, and the **AUC** is the area under it (1.0 is perfect separation, 0.5 is random guessing). The **confusion matrix** lists the four outcome counts.

Results Actually Obtained

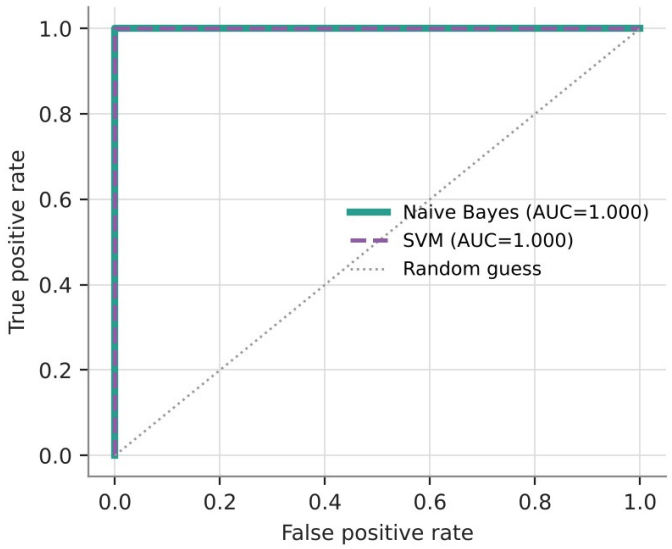
Table 9 and Figures 12 to 14 show the performance on the 2,000 held-out test emails. **Both models classified every test email correctly**: 1,201 ham and 799 spam emails, with zero false positives and zero false negatives.

ACTUAL RESULT | **Table 9.** Hold-out test-set performance (2,000 emails; spam is the positive class).

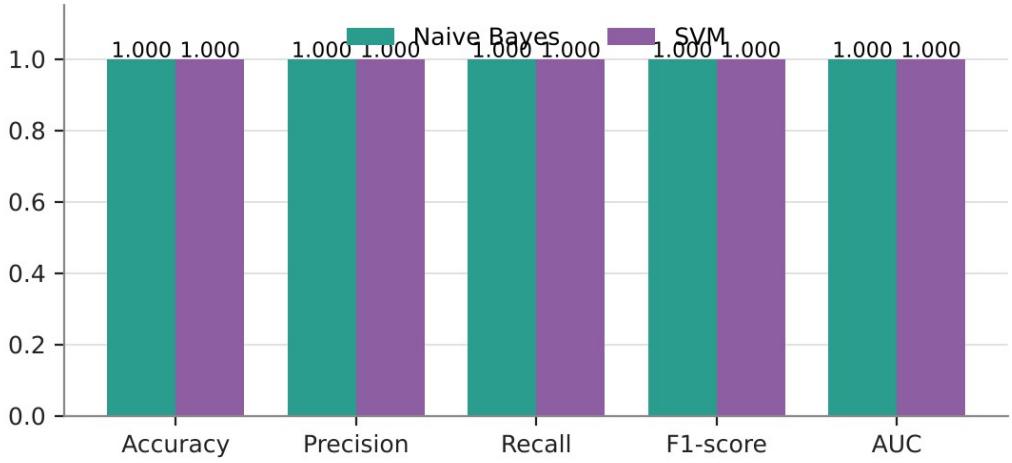
Model	Accuracy	Precision	Recall	F1-score	AUC	Errors
Naive Bayes (BoW)	100.00%	100.00%	100.00%	100.00%	1.000	0 / 2,000
SVM (TF-IDF)	100.00%	100.00%	100.00%	100.00%	1.000	0 / 2,000



ACTUAL RESULT | **Figure 12.** Confusion matrices on the 2,000-email test set (rows: actual class, columns: predicted class).



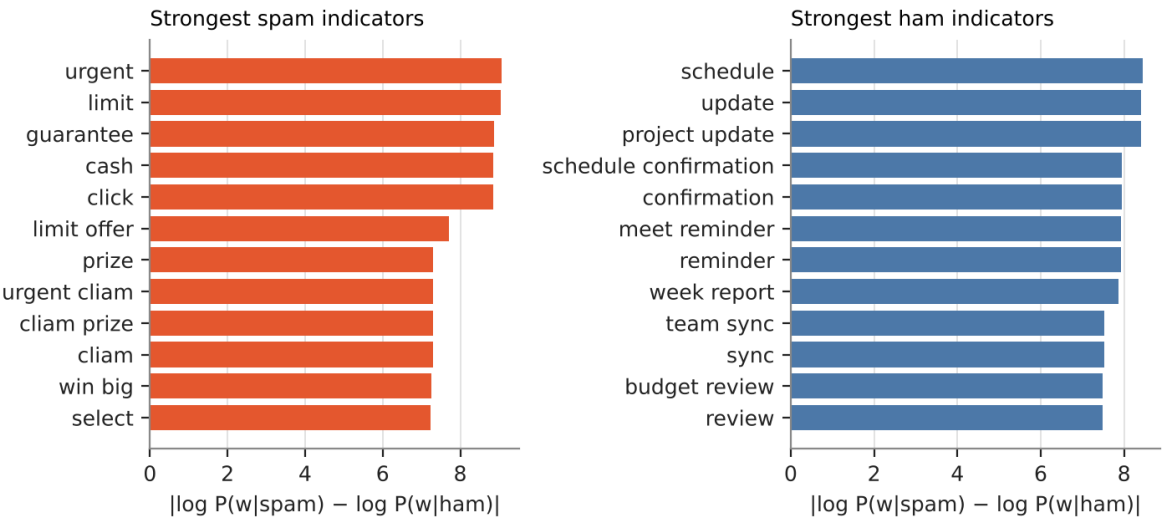
ACTUAL RESULT | **Figure 13.** ROC curves. Both models reach the top-left corner (AUC = 1.0); the Naive Bayes curve is drawn thicker so that the overlapping SVM curve stays visible.



ACTUAL RESULT | **Figure 14.** Test-set metrics of the two models.

What the models learned

Figure 15 shows the terms with the largest difference between the class-conditional log-probabilities in the Naive Bayes model. Terms such as *urgent*, *limit*, *guarantee*, *cash*, *click* and *prize* point to spam, while *schedule*, *update*, *reminder*, *confirmation*, *sync* and *review* point to ham. The SVM, trained on TF-IDF, learned very similar vocabulary; Table 10 lists its largest weights.



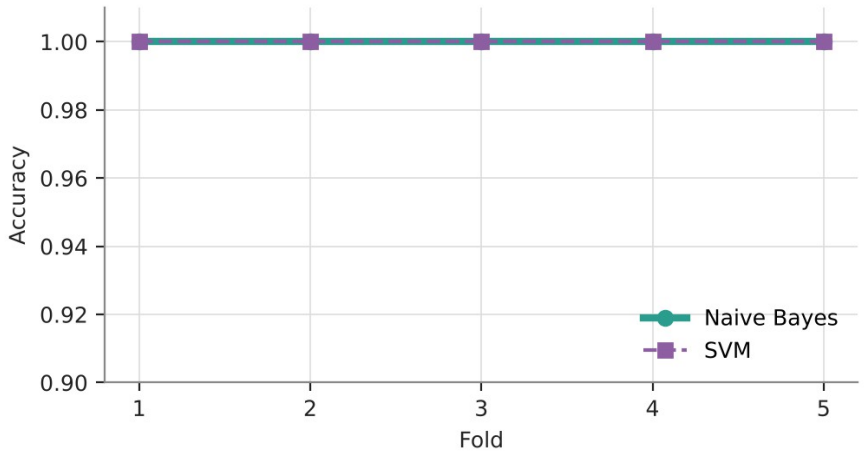
ACTUAL RESULT | **Figure 15.** Strongest spam and ham indicators in the Naive Bayes model (difference of log-probabilities).

ACTUAL RESULT | **Table 10.** Largest linear-SVM weights (positive = spam, negative = ham).

Rank	Spam term (weight)	Ham term (weight)
1	select (+1.87)	schedule (-1.25)
2	limit (+1.87)	report (-1.24)
3	urgent (+1.85)	meet (-1.09)
4	win (+1.71)	project update (-0.96)
5	guarantee (+1.63)	update (-0.96)
6	cash (+1.62)	project (-0.87)
7	click (+1.60)	meet reminder (-0.81)
8	offer (+1.58)	reminder (-0.81)

Cross-Validation and Vectorizer Comparison

A single split can be lucky, so each model was also evaluated with stratified 5-fold cross-validation on the whole dataset. Every fold produced 100% accuracy, precision, recall and F1 for both models (standard deviation 0.0). A 2×2 experiment that crosses the two vectorizers with the two classifiers also gave 100% accuracy for all four combinations (Table 11), so on this dataset the choice between Bag-of-Words and TF-IDF made no difference.



ACTUAL RESULT | **Figure 16.** Accuracy of each of the five cross-validation folds (the two lines coincide at 1.0).

ACTUAL RESULT | **Table 11.** Test accuracy for all four classifier / vectorizer combinations.

Classifier	Bag-of-Words	TF-IDF
Naive Bayes	100.00%	100.00%
Linear SVM	100.00%	100.00%

ACTUAL RESULT | **Table 12.** Wall-clock time including vectorization, measured once on a shared cloud machine (indicative only).

Model	Training time (8,000 emails)	Prediction time (2,000 emails)	Per email
Naive Bayes	210 ms	38 ms	0.019 ms
SVM	244 ms	57 ms	0.029 ms

Both models train in about a quarter of a second and predict in a few hundredths of a millisecond per email, so on a dataset of this size speed does not separate them either; Naive Bayes was slightly faster in this run, which matches its simpler training procedure.

Stress Tests: Looking Beyond the Perfect Score

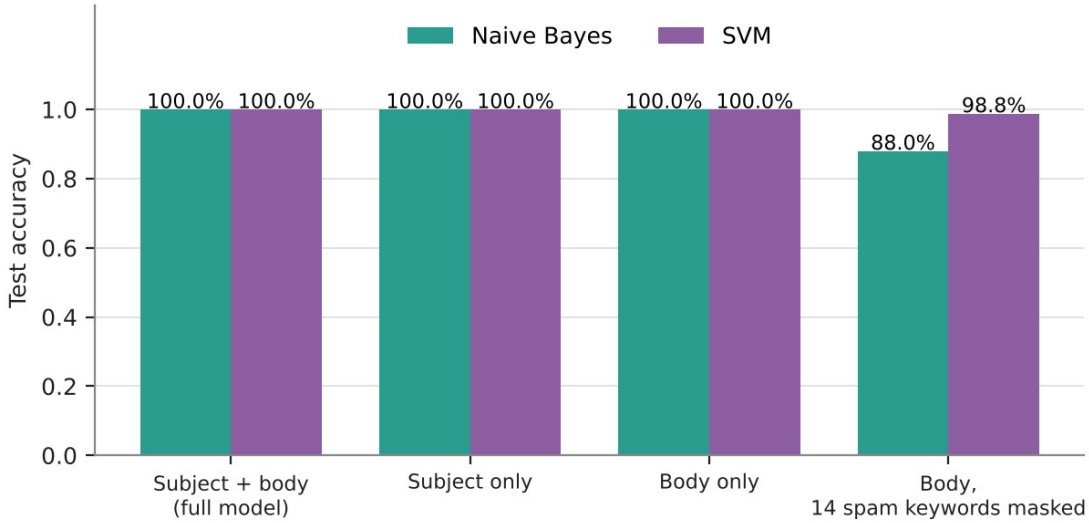
Because the perfect scores come from an easy dataset, additional experiments were designed to find out whether the two algorithms differ when the task becomes harder. All of them are real experiments on the same data and the same 80/20 split.

Which part of the email carries the signal?

The models were retrained on the subject only, on the body only, and on a *keyword-masked body*, in which the 14 most obvious spam words (big, cash, claim, cliam, click, free, guarantee, limited, now, offer, prize, selected, urgent, win) were deleted from the body before preprocessing and the subject was dropped. The masked body of the example email becomes *"Beyond baby physical environmental none meeting your foreign low and."*

ACTUAL RESULT | **Table 13.** Ablation on the text input (hold-out test set).

Input to the model	NB accuracy	NB F1	SVM accuracy	SVM F1
Subject + body (full model)	100.0%	100.0%	100.0%	100.0%
Subject only	100.0%	100.0%	100.0%	100.0%
Body only	100.0%	100.0%	100.0%	100.0%
Body, 14 keywords masked	88.0%	83.7%	98.8%	98.5%



ACTUAL RESULT | **Figure 17.** Test accuracy under different text inputs. Only the keyword-masked body separates the two models.

The subject alone and the body alone are each enough for perfect classification. Once the obvious keywords are masked, performance drops and a clear gap appears: the SVM still reaches 98.8% accuracy (precision 100.0%, recall 97.0%), whereas Naive Bayes falls to 88.0% (precision 91.3%, recall 77.3%). A plausible explanation, which was not tested directly, is that with the strong keywords gone Naive Bayes has to combine many weak word cues under its independence assumption, whereas the SVM can weight all remaining cues jointly (for example differences in the filler vocabulary or in body length) and is much less affected.

Do the metadata columns help?

ACTUAL RESULT | **Table 14.** Models that use metadata instead of, or in addition to, the text (hold-out test set).

Model (13 numeric columns, standardized)	Accuracy	Precision	Recall	F1
Gaussian Naive Bayes	86.1%	81.3%	84.5%	82.9%
Linear SVM	87.2%	84.0%	84.0%	84.0%
Linear SVM on sender_domain (one-hot) only	100.0%	100.0%	100.0%	100.0%
Linear SVM: masked-body TF-IDF + numeric columns	98.8%	99.7%	97.1%	98.4%

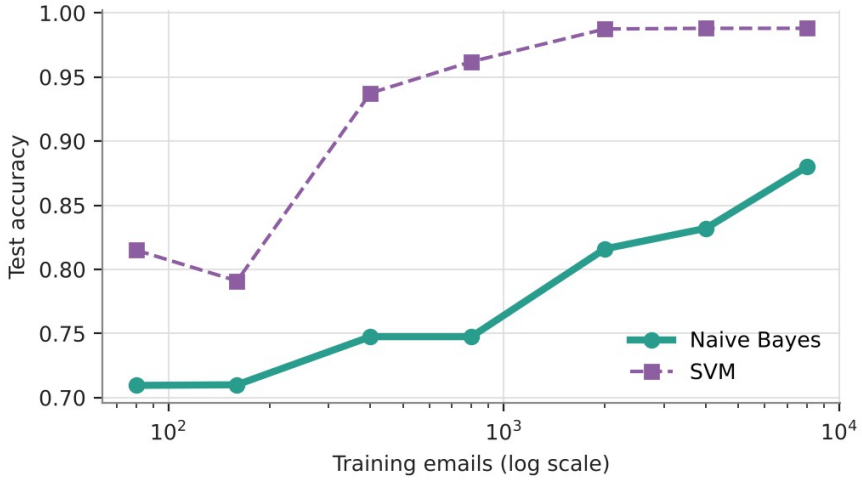
The 13 numeric metadata columns on their own give about 86–87% accuracy: useful but far from the text-based models. The sender domain alone gives 100%, which is the leakage discovered in EDA. Adding the numeric columns to the masked-body text did not improve the

SVM (98.75% against 98.8% with text alone, a difference of one email), so the text already carries almost all of the usable signal. This confirms that the abstract's choice of a text-based classifier is sensible.

How much training data is needed?

ACTUAL RESULT | **Table 15.** Learning curve on the keyword-masked body: test accuracy versus number of training emails (random subsets of the training split).

Training emails	80	160	400	800	2,000	4,000	8,000
Naive Bayes	71.0%	71.0%	74.8%	74.8%	81.6%	83.2%	88.0%
SVM	81.5%	79.1%	93.8%	96.2%	98.8%	98.8%	98.8%



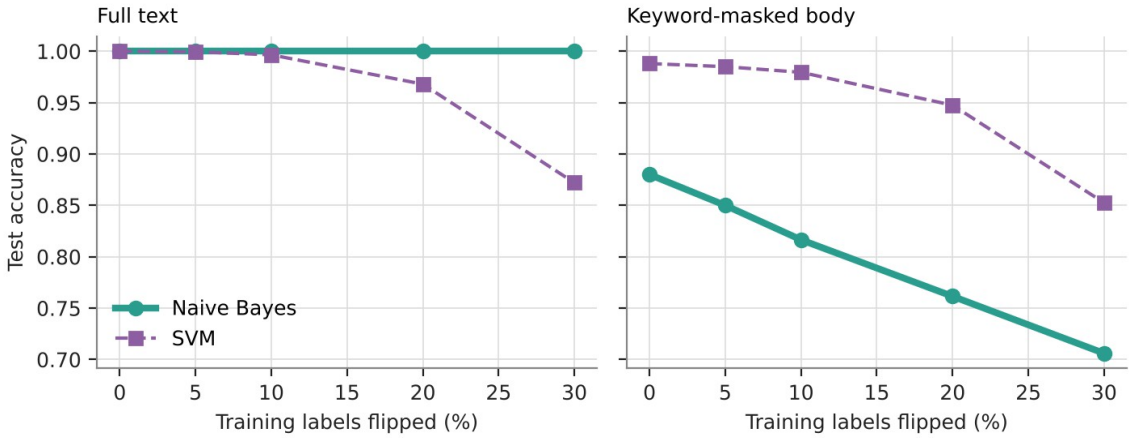
ACTUAL RESULT | **Figure 18.** Learning curves on the keyword-masked body. On the full text both models stay at 100% with as few as 80 training emails.

On the full text both models reach 100% accuracy with only 1% of the training data (80 emails). On the harder masked body the SVM learns faster and better than Naive Bayes at every size, reaching 98.8% with 4,000 emails, while Naive Bayes is still improving at 88.0% with all 8,000.

What happens when training labels are wrong?

ACTUAL RESULT | **Table 16.** Test accuracy when a share of the *training* labels is flipped at random (test labels stay correct).

Flipped training labels	NB (full text)	SVM (full text)	NB (masked)	SVM (masked)
0%	100.0%	100.0%	88.0%	98.8%
5%	100.0%	100.0%	85.0%	98.5%
10%	100.0%	99.7%	81.7%	98.0%
20%	100.0%	96.8%	76.1%	94.8%
30%	100.0%	87.2%	70.6%	85.2%



ACTUAL RESULT | **Figure 19.** Effect of label noise in the training data on test accuracy.

The results differ by input. With the full text, Naive Bayes is remarkably tolerant: it stays at 100% even when 30% of the training labels are wrong, because the correct class still wins the vote of the strong template words. The SVM, which tries to fit every training point, degrades to 87.2% at 30% noise. On the harder masked body both models degrade, but the SVM remains better throughout (for example 98.0% against 81.7% at 10% noise). No single model is more robust in every situation.

Interpreting the Results

Why 100% should not be taken at face value

A perfect score on a spam dataset is unusual. Here it is explained by the data, not by the algorithms: every subject line and every sender domain appears in only one class, spam bodies start with a handful of template words, and the filler text is random. Any reasonable text classifier solves such a task.

The correct reading is therefore: (i) the *pipeline works* (preprocessing, vectorization, training, evaluation and the web application are correct and reproducible), and (ii) the *comparison of NB and SVM* is only meaningful in the stress tests, where the SVM is more accurate when keywords are hidden and Naive Bayes is more tolerant of label noise on the full text.

The stress tests were chosen to imitate two real conditions: spammers who avoid known trigger words, and training sets that contain mislabelled mail. They are still artificial. The masking list was chosen by the team after inspecting the data, the filler vocabulary is shared by both classes, and neither test reproduces the diversity of real email. The numbers in this chapter must not be quoted as expected performance in production.

Web Application: Front End and Back End

To make the models usable, the trained objects are served by a small web application. The **back end** is a Flask application (`backend/app.py`) that loads the saved models (`nb.joblib` and

`svm.joblib`) and the metrics file at start-up and exposes the REST endpoints in Table 17. The **front end** is a single HTML page with CSS and JavaScript (`frontend/index.html`) that calls these endpoints with `fetch` and renders the answers.

SCHEMATIC | **Table 17.** REST API exposed by the Flask back end.

Endpoint	Method	Purpose
/	GET	Serves the front-end page
/api/predict	POST	Body: {subject, text}. Returns the preprocessed text and, for each model, the label, the spam probability and the eight most influential words
/api/metrics	GET	Returns dataset statistics, metrics, confusion matrices, ROC points and top words for the comparison tab
/api/samples	GET	Returns six example emails (three spam, three ham) from the dataset

```
@app.route("/api/predict", methods=["POST"])
def predict():
    body = request.get_json(force=True, silent=True) or {}
    subject, text = body.get("subject", ""), body.get("text", "")
    if not (subject.strip() or text.strip()):
        return jsonify({"error": "Please enter an email subject or body."}), 400
    clean = preprocess(f"{subject} {text}")              # same NLP as in training
    return jsonify({"preprocessed": clean,
                    "naive_bayes": run_model("naive_bayes", clean),
                    "svm": run_model("svm", clean)})
```

The page has three tabs. **Detect Spam** lets the user type or paste an email, or click one of the sample emails, and shows for each model its verdict, the spam probability, the most influential words and the preprocessed text. **Model Comparison** shows the accuracy, precision, recall and F1 of both models, the confusion matrices, ROC curves, metric bars and the strongest spam and ham words. **Dataset & Pipeline** shows the dataset statistics, the column table and the processing pipeline. To run the project: install the requirements (`python3 -m pip install -r requirements.txt`), train once with `python3 train.py`, start the server with `python3 app.py` from the `backend` folder and open the printed address (port 5001 in the team's setup, because port 5000 is used by macOS AirPlay Receiver).

Illustrative Expectations on Real-World Email

ILLUSTRATIVE ONLY: not obtained from this project

This section does not report measurements. It uses invented numbers to explain how the confusion matrix is read when a filter is imperfect, as is normal on real email corpora. None of these values come from running the code of this project.

ILLUSTRATIVE | **Table 18.** Hypothetical confusion matrix of an imperfect filter on 2,000 emails. Accuracy would be 98.2%, precision 776/788 = 98.5% and recall 776/800 = 97.0%.

	Predicted ham	Predicted spam
Actual ham (1,200)	1,188	12 (false positives)
Actual spam (800)	24 (false negatives)	776

On real, noisy corpora the literature generally reports accuracies below the perfect scores seen here, with the remaining errors concentrated in borderline messages such as newsletters and promotional mail from legitimate companies. In the hypothetical table above, 12 legitimate emails (1.0% of ham) are blocked. An administrator who regards that as too many would raise the spam threshold: this lowers false positives but lets more spam through. Both models support this trade-off, Naive Bayes through its probability output and the SVM through its decision value or the calibrated probability. How large the difference between the two algorithms would be on real data can only be answered by repeating the experiments of this chapter on a real corpus such as SpamAssassin or Enron-Spam; the code is written so that only the CSV file has to be replaced.

Discussion: Strengths and Weaknesses Observed

ACTUAL RESULT | **Table 19.** Summary of observed behaviour. Rows marked ACTUAL are results obtained in this study.

Aspect	Naive Bayes	SVM
Accuracy on the full data (ACTUAL)	100%	100%
Accuracy with 14 keywords masked (ACTUAL)	88.0%	98.8%
Label noise, 30% flipped, full text (ACTUAL)	100.0%	87.2%
Label noise, 30% flipped, masked body (ACTUAL)	70.6%	85.2%
Training / prediction speed (ACTUAL)	Slightly faster in this run	Very fast as well
Explainability	Per-word log-probability ratios	Per-word weights
Probability output	Built in	Needs calibration

Overall, the SVM is the safer default for a filter that must keep working when spammers avoid obvious keywords, while Naive Bayes remains an excellent, very light baseline that is easy to retrain and that, on this data, tolerated mislabelled training mail better when strong cues were present.

Conclusion

This case study built and evaluated a complete spam-email classification system with Naive Bayes and a Support Vector Machine. It covered NLP preprocessing (normalization, tokenization, stop-word removal, stemming), Bag-of-Words and TF-IDF feature extraction, model training and evaluation, and a Flask and browser-based application that lets a user classify an email with both models.

Answers to the research questions

- **RQ1: accuracy and false positives.** On the 2,000-email test set and in 5-fold cross-validation both models reached 100% accuracy, precision, recall and F1, with no legitimate email blocked (ACTUAL RESULT).
- **RQ2: Bag-of-Words versus TF-IDF.** All four classifier/vectorizer combinations gave 100% accuracy; the representation did not matter on this dataset.
- **RQ3: speed.** Both trained in roughly a quarter of a second and predicted in a few hundredths of a millisecond per email; Naive Bayes was slightly faster.
- **RQ4: robustness.** With 14 spam keywords masked the SVM reached 98.8% against 88.0% for Naive Bayes, and it learned faster from few examples. With 30% flipped training labels on the full text Naive Bayes stayed at 100% while the SVM fell to 87.2%. Neither model is superior in every condition.
- **RQ5: is the dataset discriminating?** No. Every subject line and sender domain occurs in only one class, so the data are separable by memorization. The perfect scores show that the pipeline works, not that spam filtering is solved.

Recommendation

For the hypothetical department of the case-study scenario, the recommended filter is the **linear SVM on TF-IDF features**, because it was the more accurate model whenever the task was made harder, with Naive Bayes kept as a light, fast baseline and a second opinion. This recommendation is conditional: it rests on stress tests with artificial perturbations, and it should be confirmed on real email before any deployment.

Limitations and future work

- Repeat all experiments on real corpora (SpamAssassin, Enron-Spam) and report confidence intervals; expect lower scores and a clearer separation of the two algorithms.
- Combine text features with trustworthy header and metadata features (sender reputation, authentication results) in a single model.

- Use a standard stemmer or lemmatizer, character n-grams to resist deliberate misspellings, and tune α and C with cross-validated grid search.
- Add a decision-threshold control and a quarantine review workflow to keep false positives close to zero.
- Evaluate adversarial robustness and concept drift (retraining schedule) and compare with logistic regression, random forests and transformer-based classifiers.

References

[1] Sahami, M., Dumais, S., Heckerman, D., and Horvitz, E. (1998). A Bayesian approach to filtering junk e-mail. AAAI-98 Workshop on Learning for Text Categorization.

[2] Drucker, H., Wu, D., and Vapnik, V. N. (1999). Support vector machines for spam categorization. IEEE Transactions on Neural Networks, 10(5), 1048–1054.

[3] Cortes, C., and Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.

[4] Joachims, T. (1998). Text categorization with support vector machines: learning with many relevant features. ECML-98.

[5] McCallum, A., and Nigam, K. (1998). A comparison of event models for Naive Bayes text classification. AAAI-98 Workshop on Learning for Text Categorization.

[6] Metsis, V., Androutsopoulos, I., and Paliouras, G. (2006). Spam filtering with Naive Bayes: which Naive Bayes? Third Conference on Email and Anti-Spam (CEAS).

[7] Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11–21.

[8] Manning, C. D., Raghavan, P., and Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

[9] Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.

[10] Graham, P. (2002). A Plan for Spam. paulgraham.com/spam.html.

[11] Pedregosa, F., et al. (2011). Scikit-learn: machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.